



Team TROOS

CV Parsing: Technical Implementation and Design Rationale

ConsultIQ - how and why the dual-mode CV parsing system was built this way

1. Purpose of This Document

CV parsing is one of the most technically involved features in ConsultIQ, and also one of the easiest to underestimate from the outside - “read a CV and fill in a form” sounds simple until the actual variety of real-world CVs, the trade-offs between different technical approaches, and the compliance obligations around personal data are taken into account. This document exists to make that complexity visible: what was built, how it works end to end, and - just as importantly - why each significant decision was made rather than an alternative.

It is written for two audiences at once. A technical reader (a marker, a future team member, a developer picking this up later) should be able to trace exactly how a CV moves through the system, which file does what, and why. A non-technical reader (a client, a project sponsor, an examiner without a software background) should be able to read the plain-English explanations woven through each section and come away understanding what the system does, why it took real engineering effort, and why the two-mode design is a deliberate, evidence-based decision rather than an arbitrary one.

This document covers the CV parsing feature specifically. Data privacy and POPIA-specific commitments are documented separately in the CV Parsing Dual-Mode Design and POPIA Compliance Notice; this document cross-references that one rather than repeating it.

2. Executive Summary

In plain terms: when a Consultant Manager uploads a consultant's CV, ConsultIQ can read it automatically and pre-fill the consultant's profile - their skills, work history, certifications, and education - instead of someone typing all of it in by hand. That sounds like a small convenience. Building it properly turned out to require solving several genuinely hard problems at once.

The first hard problem is that there is no single “correct” way to read a CV automatically. A rule-based approach (pattern matching against a fixed template) is fast, free, and keeps all data inside ConsultIQ's own infrastructure - but it can only reliably read CVs that follow a format it was specifically built to expect. A modern AI model can read almost any CV in almost any format - but it costs a small amount per use and means sending a CV's content to an external service. Rather than picking one and living with its weaknesses, ConsultIQ offers both, as a deliberate, disclosed choice made at the point of upload. That decision, and the research behind it, is the subject of Section 4.

The second hard problem is that both of these approaches, however different internally, need to produce identical, trustworthy output that the rest of the system can rely on - and neither one is ever allowed to update a consultant's profile without a human checking it first. Sections 6 through 10 explain exactly how that works, file by file.

The third thing worth knowing going in: this feature was built, broken, debugged, and rebuilt several times over as real technical problems surfaced - a PDF form's fields not appearing where a text-reading library expected them, a dropdown field silently reporting a default

value nobody selected, a JavaScript proxy subtly breaking database connectivity in a way that took real investigation to trace. Section 13 documents these honestly, not to pad this document, but because they are a genuine, accurate record of the depth of engineering this feature actually required - the kind of problem that does not show up by reading the finished code, only by having built it.

3. Why CV Parsing Exists At All

ConsultIQ's core purpose is matching consultants to project requirements based on their skills, experience, and availability. That matching engine is only as good as the data behind it — and manually typing in a consultant's full skill set, work history, and certifications is slow, error-prone, and a real barrier to onboarding a new consultant quickly. Requirement R7 of the Software Requirements Specification exists specifically to remove that barrier: allow a CV to be uploaded and have the system extract that data automatically, with a human always reviewing the result before it becomes part of a real profile.

That last point - a human always reviews the result - is not a minor implementation detail. It is the single design constraint that everything else in this document is built around. No extraction, from either path, is ever trusted enough to write directly to a consultant's profile unconfirmed.

4. The Central Research Question: Rule-Based vs. AI-Assisted

4.1 The Original Constraint

CV parsing was first built under a real project constraint: no external AI API integrations were permitted for the earlier demo milestones. Under that constraint, the only viable approach was rule-based - regular expressions matching known patterns, and a curated dictionary of recognised skills. That work was not wasted; large parts of it (the template design, the confidence-scoring approach, the validation logic) survive in the final system, described in Section 7.

Once that constraint was lifted, the team treated the choice between rule-based and AI-assisted extraction as a genuine engineering decision to research properly, rather than simply defaulting to whichever seemed more modern. That research is summarised below.

4.2 What the Research Found: Three Generations of CV Parsing

Industry analyses of resume and CV parsing describe its evolution in three broad generations, each with a real, documented accuracy ceiling:

- Generation 1: rule-based / keyword matching: pattern matching and fixed keyword lists, typically topping out around 60 -70% field-level accuracy, and considerably worse on any CV that doesn't follow an expected layout.

- Generation 2: machine-learning / named-entity recognition: models trained on large resume datasets, reaching roughly 85 - 92% accuracy on formats similar to their training data, with real drops on novel or unusual layouts.
- Generation 3: large language models: better handling of long-range context, unusual phrasing, and formats the system has never explicitly seen before, at the cost of non-determinism and the (rare, but real) possibility of a confidently wrong answer.

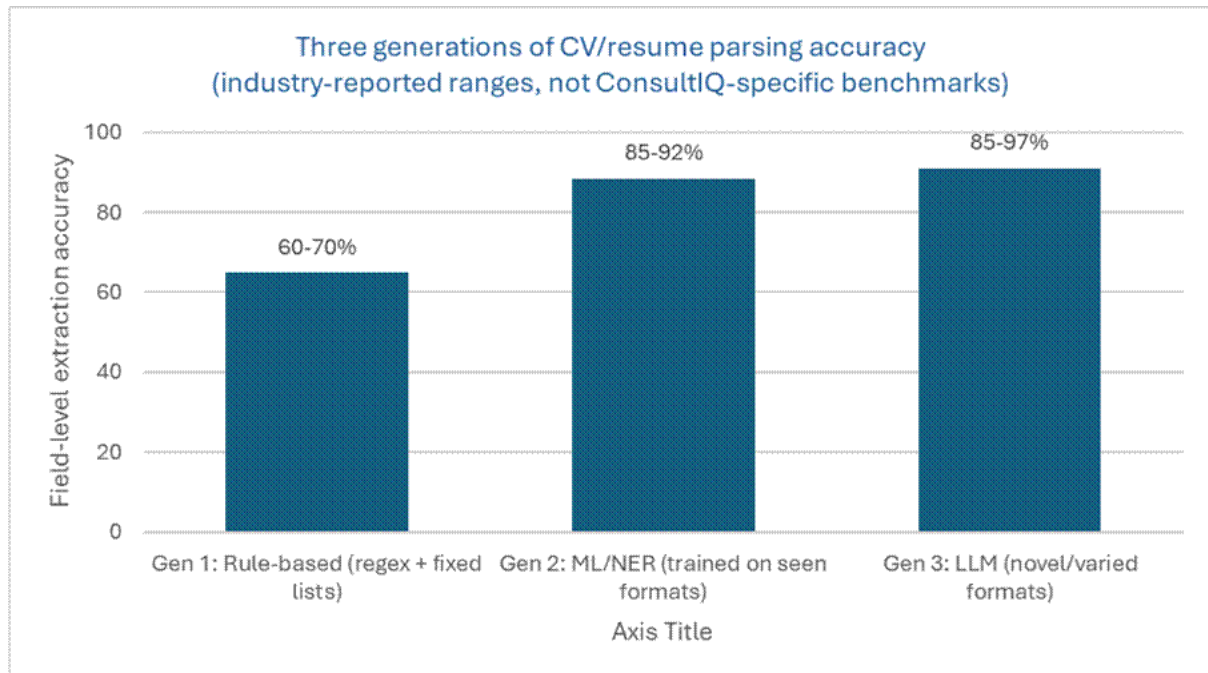


Figure 1 - Approximate field-level accuracy ranges across the three generations of CV/resume parsing technology, as reported across the industry sources cited in Section 15. These are general industry ranges, not a benchmark run against ConsultIQ's own parser.

The practical takeaway: ConsultIQ's rule-based parser is architecturally a first-generation system. It is genuinely good at what it is built to do - reading a CV that follows ConsultIQ's own template - but it was never going to reliably read an arbitrary CV a consultant already has sitting on their computer. That gap is precisely what motivated the research into an AI-assisted alternative, rather than simply trying to write more regular expressions.

4.3 The Nuance That Shaped the Final Design

The research did not point to a simple "AI is always better" conclusion, and it would have been a mistake to treat it as one. A peer-reviewed comparison published in JAMIA Open directly tested regular-expression extraction against a large language model on a real, high-volume task: pulling a standardised score (the BI-RADS breast-imaging score) out of 7,764 real radiology reports. The result: regex achieved 89.20% accuracy versus 87.69% for the LLM - a difference the study found was not statistically significant - while completing the task roughly 28,000 times faster.

The explanation the study itself gives is the important part, not just the numbers: BI-RADS scores are a standardised, tightly-formatted piece of information, which is exactly the

situation regular expressions are naturally suited to. The advantage of a large language model shows up on unstructured, highly variable text - not on a narrow, well-defined field that always appears in a predictable shape.

That distinction maps directly onto ConsultIQ's own design. An email address or a phone number is a BI-RADS-style field - narrow, well-defined, and something regular expressions already handle close to their practical ceiling. A CV's work-history description, written in free prose by a different person every time, is the opposite case - exactly where the research says a large language model earns its cost. This is part of why the rule-based path's regex validators for email, phone, and date format were kept even in the final design, rather than assumed to need replacing wholesale.

4.4 The Cost Question

Official Anthropic API pricing puts Claude Haiku 4.5 at \$1 per million input tokens and \$5 per million output tokens, with Claude Sonnet 5 at \$2 / \$10. A typical one-to-two-page CV runs to roughly 1,500–2,500 tokens of text, with perhaps 500–800 tokens of structured data returned - which works out to approximately half a cent to one cent per CV processed. At that price, cost was never the deciding factor between the two approaches; the deciding factors were accuracy, format flexibility, and what happens to a consultant's data.

4.5 A Third Option Considered, and Set Aside

Dedicated, purpose-built resume-parsing APIs exist as a commercial category - Affinda, Textkernel (which absorbed the long-standing Sovren parser in 2023), RChilli, DaXtra, and others. These vendors report strong accuracy figures and handle a very wide range of CV formats and languages out of the box. They were considered and set aside for two concrete reasons: cost, and vendor scope.

Published 2026 self-serve pricing for this category runs roughly \$75–\$150 per month for a few hundred to a few thousand parses, or in the range of \$40–\$200 per 1,000 CVs - meaningfully more expensive than a direct LLM API integration at ConsultIQ's expected volume, for a system whose whole value proposition (per Section 4.4) is that the per-CV cost is already close to negligible. Beyond cost, adopting one of these vendors would mean introducing an entire second external dependency and vendor relationship purely for CV parsing, on top of the Anthropic API already being adopted - complexity the research did not justify for the accuracy gain on offer.

4.6 Why Anthropic's Commercial API, Specifically

Anthropic's Trust Center states directly that, by default, commercial products - including the Anthropic API, as distinct from Anthropic's consumer chat products - are not used to train Anthropic's models, and are only used this way if a customer explicitly opts in. This is not a minor detail: it is the factual basis for the consent language shown to consultants on ConsultIQ's own POPIA disclosure page, and it is the reason the API (not a consumer AI product) was the only option ever seriously considered for a feature that processes real personal data.

4.7 The Decision

Rather than replacing the rule-based parser with an AI-assisted one, ConsultIQ offers both, selectable per upload, defaulting to rule-based (the option that shares no data externally) and requiring an active, informed choice to use AI-assisted parsing instead.

This mirrors what the research itself recommends: the JAMIA Open study's own authors conclude by pointing toward hybrid approaches that combine regex precision on structured elements with LLM understanding on complex, unstructured information - the same underlying principle behind offering both paths rather than forcing a single one-size-fits-all answer.

5. System Architecture at a Glance

Before the detailed routing logic, it helps to see the shape of the whole pipeline a CV moves through - from the moment a Consultant Manager uploads a file to the moment a result is ready for review. This diagram deliberately stays at a high level; the decision point in the middle (labelled here as the orchestrator) is expanded in full in Section 6.

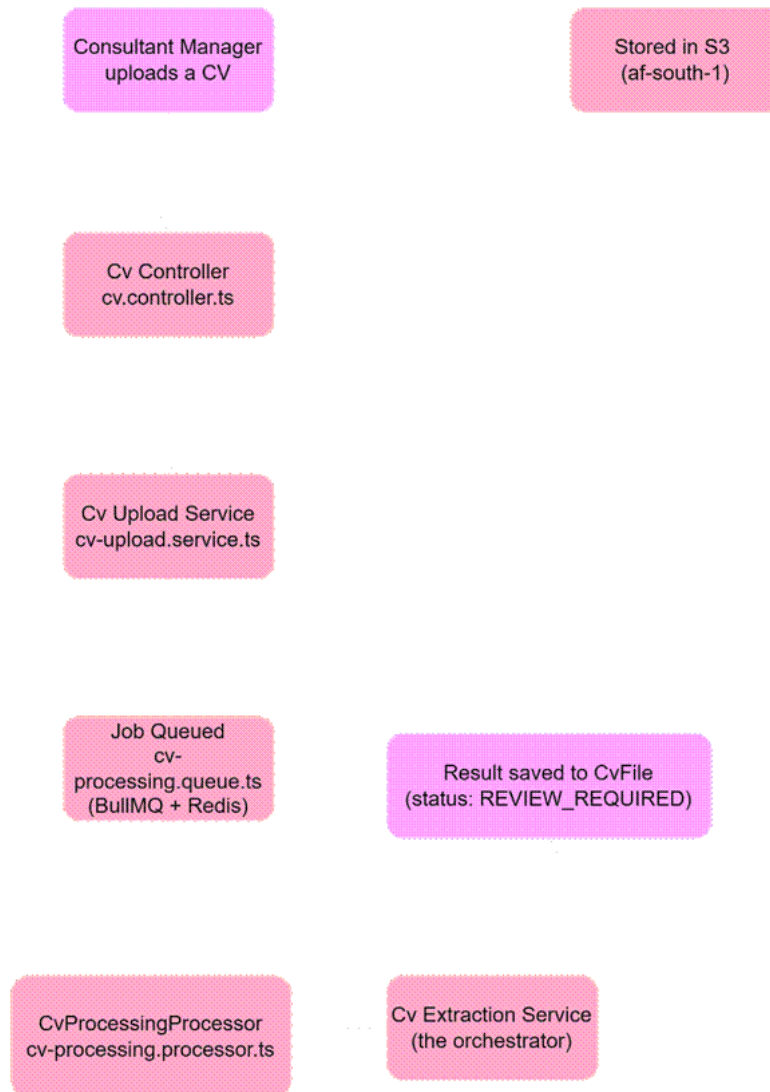


Figure 2 - The complete CV upload pipeline, independent of which parsing mode is eventually chosen.

Two design choices are worth calling out here even at this high level. First, the work happens asynchronously, behind a job queue (BullMQ, backed by Redis) - uploading a CV returns to the Consultant Manager immediately rather than making them wait for extraction to finish, which can take anywhere from under a second (rule-based) to several seconds (AI-assisted, including retries). Second, the file itself is stored in Amazon S3 in the af-south-1 (Cape Town) region specifically - a POPIA-relevant choice covered in the compliance document - before any parsing ever begins.

6. The Routing Decision, In Full Detail

This is the heart of the system: once a queued job reaches the orchestrator (CvExtractionService), what actually happens depends entirely on which parsing method was chosen for that specific upload, stored on the CvFile record itself. The diagram below traces both branches completely — every service involved, the exact TypeScript file it lives in, and both the success and failure paths.



Figure 3 - The full decision-and-execution flow for both parsing routes, including failure handling. Rectangular boxes are processing steps; diamonds are decisions; the gold box is the point where both paths converge on identical, shared logic.

Reading the diagram in plain English

Every CV extraction job starts the same way, regardless of which mode was picked: the record's status is set to PROCESSING, and the orchestrator checks the parsingMethod field on the database record. From there, the two paths genuinely diverge - they do not share a code path again until validation, near the end.

If rule-based was chosen, the very first check is whether the uploaded file is actually a PDF - rule-based mode has no equivalent of a fillable form for a Word document, so a non-PDF upload is rejected immediately and honestly, rather than attempting to force it through a path that cannot work. A valid PDF is then handed to a service that reads the document's named form fields directly - not by reading the page as text, but by asking the PDF for the exact value of a field named, for example, skill_1_name. That raw data then passes through normalisation logic: matching typed skill names against a curated dictionary (including common aliases like “Golang” for “Go”), validating that dates, emails, and phone numbers are actually well-formed, and computing a confidence score for every extracted value.

If AI-assisted was chosen instead, the file (PDF or Word document) is first converted into plain text, then that text is sent to Claude Haiku 4.5 with a strict schema forcing it to return exactly the fields the system expects, in exactly the shape it expects them - not a prompt asking nicely for JSON and hoping, but a mechanism that makes a malformed response structurally impossible to receive. If the model's response doesn't fit that shape, or the API call itself fails for a recoverable reason (a rate limit, a timeout), the system retries automatically up to three times with increasing delays before giving up.

Whichever path produced a result, the exact same validation logic runs next - the same email, phone, and date checks, applied identically regardless of origin. This is a deliberate, load-bearing design decision, covered in Section 10.1: the system was built so that nothing downstream of this point ever needs to know or care which parser actually ran.

7. Deep Dive: The Rule-Based Path

7.1 Why a Fixed Template, Not Free-Form Parsing

An earlier version of the rule-based parser attempted to read a CV's free-form text and guess where each section began, using regular expressions to recognise section headings like “Work Experience” or “Skills” written in any of dozens of common phrasings. This works, but only within real limits: a heading regex built to catch the word “experience” can just as easily match that word appearing inside an ordinary sentence elsewhere on the page, and the entire approach is fundamentally guessing at structure that was never guaranteed to exist. The team made a deliberate decision to remove that guesswork entirely by defining a

fixed CV template with an unambiguous, locked structure - turning “where does the Skills section start” from an inference problem into a non-problem.

7.2 The Fillable PDF Template

The template is a genuine interactive PDF form, not a document with instructions to follow. Section headings, field labels, and structural separators (the “|” and “-” characters between fields) are locked, non-editable page content; only the boxed fields themselves can be filled in. Job Type and Work Model are dropdown fields constrained to the exact enum values the system expects (FULL_TIME, PART_TIME, and so on) rather than free text - removing an entire category of possible typo or synonym mismatch before it can happen. This was a deliberate choice over a plain Word document specifically because a Word document offers no way to stop someone editing the very structure the parser depends on.

7.3 A Real Discovery: Why Text Extraction Doesn't See a Filled Form

Worth understanding, not just noting:

Before trusting this design, it was tested empirically rather than assumed to work: a copy of the template was filled with sample data and run through the same text-extraction library (pdf-parse) already used elsewhere in the system.

Every filled-in value: the name, the email, every skill - was completely absent from the extracted text. Only the static labels and headings came through.

The reason: a PDF form stores what someone types into a field in a separate part of the file (the form's own internal structure) from the page's visible text content. A tool built to read “the text on the page” has no reason to also read “the values inside this form's fields” - they are, technically, two different things living in the same file.

This discovery changed the actual implementation: rather than extracting text and pattern-matching it, the rule-based path reads the PDF's form fields directly, by their exact field names, using a different library (pdf-lib) built specifically for that purpose. This is not a workaround - it turned out to be a better approach than text extraction ever could have been for this specific input, since asking a form directly for the value of a field named skill_1_name has no ambiguity to resolve at all.

7.4 Reading the Form: CvFormReaderService

The file `cv-form-reader.service.ts` is responsible purely for extracting raw values from the filled PDF - it performs no interpretation, normalisation, or scoring of its own. One subtlety it specifically guards against: a dropdown field that was never touched by the person filling in the form still reports its default selected value (for example, FULL_TIME), rather than reporting nothing. If this were trusted blindly, every unfilled “role” slot on the template - someone with only two jobs listed, out of the six available - would appear to the system as a real, filled-in job with a fabricated type. The fix: presence of a whole entry (a role, a

certification, an education record) is judged by whether its required text field (a job title, a certification title) is actually filled in, never by the dropdown fields alone.

7.5 Making Sense of the Data

Three files work together once raw values have been read:

- `skills.dictionary.ts`: a curated list of recognised skills, grouped by category (languages, frontend, backend, cloud/DevOps, and more), plus a table of common aliases (“JS” → “JavaScript”, “Postgres” → “PostgreSQL”) so an informally-typed skill name still resolves correctly rather than being flagged as unrecognised.
- `section-patterns.ts` : not a section-finder any more (Section 7.3 made that unnecessary) but a set of format validators: does this look like a real email address, a valid South African phone number, a correctly formatted date. These exist to catch a malformed value the consultant may have mistyped, not to discover data hiding in free text.
- `cv-parsing.service.ts`: the orchestrating logic that ties the above together: normalising each typed skill against the dictionary, validating every date and contact field, and computing a confidence score for every value - lower for anything that didn't cleanly match or validate, so a manager reviewing the result can see at a glance what deserves a second look.

7.6 Known, Documented Limitations

These are deliberate, disclosed trade-offs, not oversights:

- The skills dictionary is scoped to software and IT consulting - matching BBD's own domain - for this initial release. A CV in a different industry may see more skills flagged as unrecognised.
- The template has a fixed capacity (10 skills, 6 roles, 6 certifications, 6 education entries). A consultant with more entries than this needs AI-assisted mode instead, which has no such limit.
- An unmatched skill is currently flagged with a low confidence score rather than routed through a formal approval workflow, since the shared, Admin-managed skills reference list that requirement R7.2.6 ultimately calls for does not yet exist elsewhere in the system. This is recorded as open follow-up work, not a silent gap.

8. Deep Dive: The AI-Assisted Path

8.1 Why Claude Haiku 4.5

Haiku 4.5 was chosen specifically because structured field extraction from a page of text is not a task that benefits from a larger, more expensive reasoning-focused model - it benefits from reliability and low cost at volume. At roughly half a cent per CV (Section 4.4), the cost

difference between models was never the constraint; Sonnet 5 remains available as a fallback if real-world testing shows Haiku struggling specifically on heavily designed or graphically unconventional CVs.

8.2 Structured Extraction, Not a Hopeful Prompt

The system does not ask Claude to “please respond in JSON” and parse whatever comes back. `cv-extraction.schema.ts` defines a strict tool schema: every field, its type, and (for fields like job type and work model) the exact enum values allowed - and the API call forces the model to respond by calling that tool. A response that doesn't match the schema is not a plausible outcome the code has to defensively handle; it is not a valid response at all.

`cv-extraction.prompt.ts` carries the actual extraction rules: normalise dates to a single ISO format, treat an ongoing role or qualification as an absent end date rather than inventing today's date, infer skills implied by a job description and not just an explicit skills list, and - critically - keep every confidence score honest: a value taken verbatim from a clearly labelled section should score near certain, while anything pieced together from context should score lower rather than being reported with false confidence.

8.3 Resilience: What Happens When Something Goes Wrong

The file `claude-extraction.service.ts` distinguishes carefully between two categories of failure, because treating them the same would either waste time or hide a real problem. A rate limit or a temporary network timeout is retried automatically, up to three attempts with an increasing delay between each. An invalid API key, a malformed request, or a billing issue is not retried at all - retrying an error that will fail identically every time wastes time without ever succeeding, so the system fails fast and reports clearly instead.

8.4 The Competency Signal: A Capability Only This Path Offers

Claude's extraction includes something the rule-based path structurally cannot offer: for every skill it identifies, it also provides a reasoned, human-readable judgement of the consultant's likely competency level, with an explanation citing specific evidence from the CV - years of experience, project scope, seniority of the roles involved. This is never written to a consultant's profile automatically; per requirement R2.1.3, the actual saved competency level is always computed from a formula using the years of experience and a confidence rating the Consultant Manager selects manually during review. Claude's competency judgement exists purely as helpful, transparent context shown alongside the review screen - advisory, not authoritative, and clearly presented as such.

9. The Shared Contract: One Data Shape, Two Origins

However differently the two paths work internally, both are required to return data in exactly the same shape - a design decision that is easy to underestimate but genuinely central to the whole system's maintainability. Every piece of code downstream of extraction - the

validator, the status logic, eventually the review screen - is written once, against one shape, and works identically no matter which parser produced the data.

A subtle but important naming decision:

Early in development, a single field on each extracted skill was overloaded to mean two different things depending on context: the parser's own certainty that it read a value correctly, and a completely separate, human-entered confidence rating used elsewhere in the system for a different purpose (computing competency level).

Sharing one name across two unrelated meanings is exactly the kind of subtle bug that surfaces only much later, when someone wires the wrong value into the wrong formula. The fix was a rename - `extractionConfidence` for the parser's own certainty - making that category of mistake structurally impossible rather than merely documented against.

The one place the two paths are allowed to differ is `competencySignals` - present only on the AI-assisted path's output, and treated everywhere downstream as optional, advisory context rather than something the rule-based path is expected to fake an equivalent for.

10. Shared Safeguards, Regardless of Path

10.1 Field Validation: One Check, Either Origin

`cv-field-validator.service.ts` runs the same email, phone, and date-format checks against the output of either parser, producing specific, actionable warnings ("contact.email does not look like a valid email address") rather than a vague overall confidence number. This is the file that most directly embodies Section 4.3's research finding: regular expressions remain the right tool for checking a narrow, well-defined field, and that stays true regardless of which parser originally produced the value.

10.2 Human Review, Always

No extraction, from either path, is ever written to a consultant's profile without a Consultant Manager reviewing and explicitly confirming it first. A failed extraction leaves the record in a clean FAILED state rather than a partial or silently incorrect one. This single rule is the actual safety net underneath everything else described in this document.

10.3 Data Privacy and Encryption

The full data-privacy and POPIA-compliance rationale - including the AI-assisted opt-in consent mechanism, Anthropic's data-handling commitments, and the field-level encryption applied to stored CV text - is documented in full in the companion CV Parsing Dual-Mode Design and POPIA Compliance Notice, to avoid duplicating and risking drift between two documents describing the same commitments.

11. Design Patterns and Architectural Decisions

Strategy Pattern

The choice between rule-based and AI-assisted extraction is a textbook application of the Strategy pattern already used elsewhere in ConsultIQ (Section: Design Patterns, main SRS): the orchestrator selects a parsing strategy based on a single stored value (parsingMethod) rather than branching internally with duplicated logic, and either strategy can be extended, replaced, or added to independently as long as it honours the same shared contract described in Section 9.

Shared Contract / Common Interface

CvParsingResult acts as a common interface both heterogeneous implementations must satisfy - conceptually related to the Adapter pattern, in that it lets two structurally different subsystems (a regex-and-forms engine, an LLM API client) present an identical face to everything that consumes their output.

Pipes and Filters

The extraction pipeline itself - read or extract, then normalise, then validate, then persist - mirrors the Pipes and Filters style already governing ConsultIQ's event-driven architecture (per the main SRS's architecture section), with each stage a self-contained, independently testable unit that passes its output to the next.

Fail-Safe Defaults

The parsing-method toggle defaults to rule-based - the option that shares no data externally - requiring an active, informed choice to opt into AI-assisted parsing rather than an active choice to opt out of it. The database column itself carries the same default, so even a code path that forgets to specify a method explicitly still fails safe rather than silently defaulting to the data-sharing option.

Chain of Responsibility

The AI-assisted path's retry logic is a small but real application of this pattern: each attempt either succeeds, fails in a way that ends the chain immediately (a non-retryable error), or fails in a way that passes control to the next attempt in the chain, up to a fixed limit.

12. Engineering Rigor: Testing Strategy

Every file described in this document has a dedicated, passing automated test suite - the PDF form reader, the rule-based parser, the shared validator, the Claude extraction service, and the orchestrator that ties them together. This was treated as non-negotiable throughout, not an afterthought added at the end.

Some of this testing required genuine care to get right, not just volume. Mocking the Anthropic SDK, for instance, required preserving its real error classes so that instanceof checks in the retry logic could still be tested meaningfully - a naive mock would have silently made every "does this error get retried correctly" test meaningless. Mocking a filled PDF form required simulating pdf-lib's actual behaviour closely enough to catch the dropdown-default-value issue described in Section 7.4 in a test, not just in production. The

orchestrator's own test suite deliberately verifies both success and failure for both parsing paths independently, plus the shared validation and persistence logic that runs regardless of which path executed.

13. Real Problems Found and Solved Along the Way

These are included deliberately and honestly, because they are the clearest evidence of how much genuine engineering depth this feature required - none of them are visible just by reading the finished code.

The text-extraction discovery (Section 7.3)

Empirically testing the assumption that a filled PDF form's values would appear in extracted text — rather than simply trusting it - surfaced a fundamental architectural problem before it could reach production: the values were completely absent, every time, regardless of extraction library used.

The dropdown default-value trap (Section 7.4)

A PDF dropdown field with a default selection reports that default even when nobody touched it - discovered and fixed before it could silently fabricate job types on every unfilled role slot in the template.

The Prisma extension prototype-chain bug

Adding field-level encryption to the database layer required wrapping the Prisma client in a way that preserves NestJS's lifecycle hooks (connecting and disconnecting cleanly). The first, seemingly reasonable fix broke the underlying database client's own methods in a way that only surfaced as a runtime crash on startup; the actual fix required understanding a genuinely subtle interaction between JavaScript's Proxy objects and prototype chains, verified with isolated test scripts before being trusted in the real service.

Migration and CI environment drift

Several rounds of database migration history diverging from the actual database state - and, separately, a CI pipeline missing an environment variable the encryption layer needed - required careful, methodical diagnosis rather than guesswork, given the real risk of data loss from an incorrect fix.

None of these were signs of things going wrong. They are exactly the kind of problem that only surfaces when a system is being built and verified properly rather than assumed to work - which is, ultimately, the actual argument for why this feature represents as much engineering effort as it does.

14. Known Limitations and Future Work

- The rule-based skills dictionary is scoped to software/IT consulting; broader industry coverage is a deliberate future extension, not an oversight.

15. Sources and Research References

Every source below was read and genuinely weighed as part of the decision documented in Section 4 - not added after the fact to justify a decision already made.

CV/resume parsing accuracy and methodology

“AI Resume Parsing in 2026: How It Works, How Accurate It Actually Is, and What Breaks It” — TheHireHub —

<https://www.thehirehub.ai/blog/ai-resume-parsing-in-2026-how-it-works-how-accurate-it-actually-is-and-what-breaks-it>

“The Evolution of Resume Parsing: Past, Present, Future” — Recrew.ai —

<https://www.recrew.ai/blog/evolution-of-resume-parsing-technology-past-present-and-future>

Sippo et al., cited via “A Comparative Performance Analysis of Regular Expressions and an LLM-Based Approach to Extract the BI-RADS Score from Radiological Reports” - JAMIA Open (Oxford Academic) <https://academic.oup.com/jamiaopen/article/8/6/ooaf128/8322185>

Dedicated resume-parsing vendor landscape (considered and set aside — Section 4.5)

“12 Best Resume Parser APIs in 2026 (Compared & Priced)” — Eden AI —

<https://www.edenai.co/post/best-resume-parser-apis>

“Resume Parsing Tools: 9 Best Options for 2026” - Pin -

<https://www.pin.com/blog/best-resume-parsing-tools/>

“Best Resume Parser Software 2026: 12 Tools” - Truffle -

<https://www.hiretruffle.com/blog/resume-parser-software>

Anthropic / Claude API: pricing, capabilities, and data policy

Anthropic API Pricing: Claude Platform Documentation -

<https://platform.claude.com/docs/en/about-claude/pricing>

Anthropic Trust Center FAQ: commercial product data-training policy -

<https://trust.anthropic.com/faq>

Anthropic Privacy Center : Commercial Customers -

<https://privacy.claude.com/en/collections/10663361-commercial-customers>

Anthropic Privacy Center: “Is my data used for model training?” -

<https://privacy.claude.com/en/articles/7996868-is-my-data-used-for-model-training>

Implementation references

prisma-field-encryption - npm package documentation -

<https://www.npmjs.com/package/prisma-field-encryption>

Prisma - official documentation on resolving migration drift and baselining an existing database -

<https://www.prisma.io/docs/orm/prisma-migrate/workflows/patching-and-hotfixing>

16. Glossary

Term	Plain-English meaning
Rule-based parsing	Extracting data by matching fixed patterns and a known template - fast, free, and entirely internal, but only works on CVs shaped the way it expects.
AI-assisted parsing	Extracting data using Claude Haiku 4.5 to read and understand a CV's content - handles far more formats, at a small cost per CV and by sharing CV text with Anthropic's API.
Orchestrator	The piece of code (CvExtractionService) that decides which parsing method to run and manages the process from start to finish.
Schema-forced extraction	Requiring Claude to respond in one exact, pre-defined data shape, so a malformed response is structurally impossible rather than merely unlikely.
Confidence score	A number reflecting how certain the system is that a piece of extracted data is correct - lower for anything guessed, inferred, or not cleanly matched.
Competency signal	An AI-generated, advisory judgement of a consultant's skill level with reasoning attached - shown to a manager for context, never saved automatically.
extractionStatus	The current stage of a CV's processing: PENDING, PROCESSING, REVIEW_REQUIRED, or FAILED.